

UDP og realtidsspil

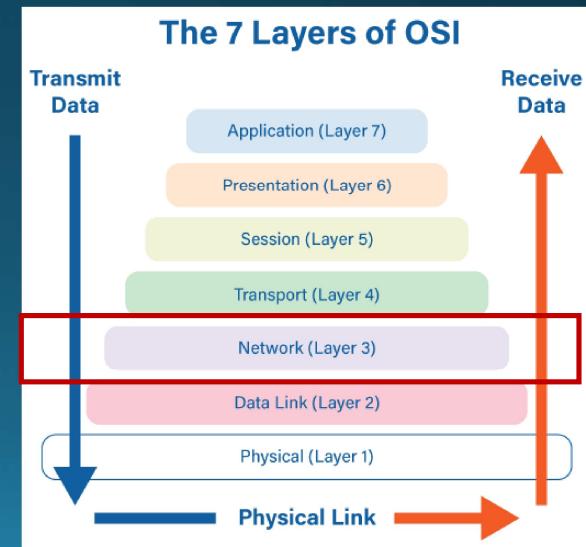
Netværk og distribueret design

Dagsorden

- Valg af protokol
- UDP
- TCP vs UDP
- Øvelse UDP
- Realtidsspil med UDP
- "Competitive" øvelse
- UDP Realtids øvelse (kompleks)
 - Resten af dagen + i morgen
 - Alternativt, arbejde videre med tidligere øvelser
 - Plus se opsummerings videoer på moodle

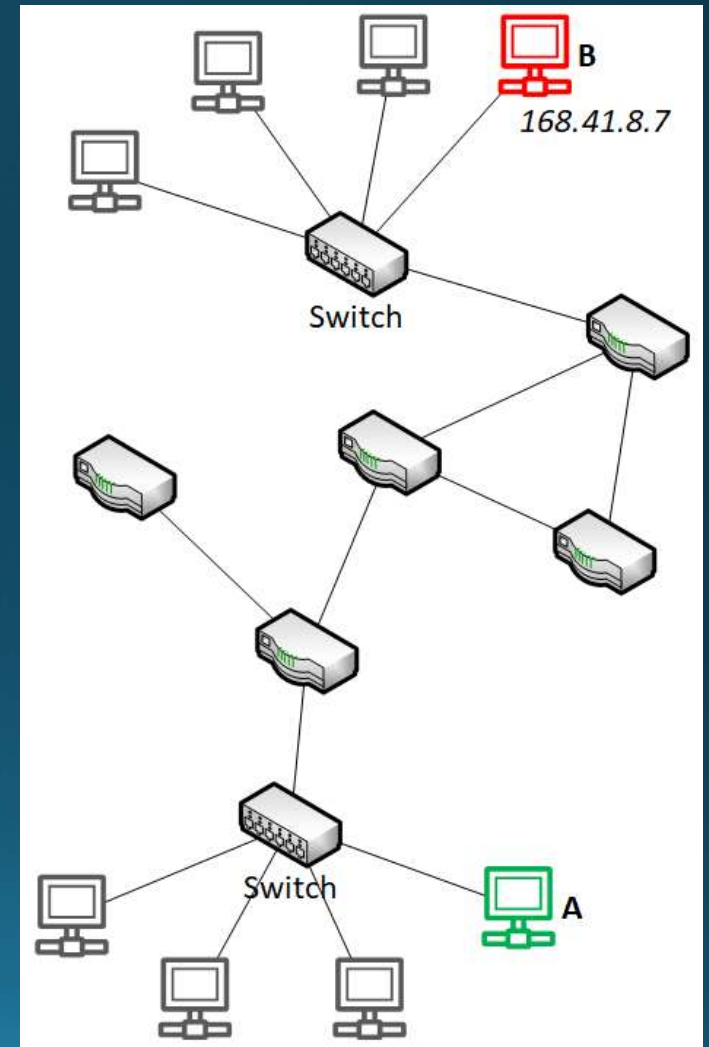
Valg af protokol

- Netværkslaget sørger for kommunikation mellem "hosts"
 - Vha. IP protokollen
- Transportlaget håndterer kommunikation mellem processerne på de "hosts"
 - Enten gennem TCP eller UDP protokollen
 - Vha. porte



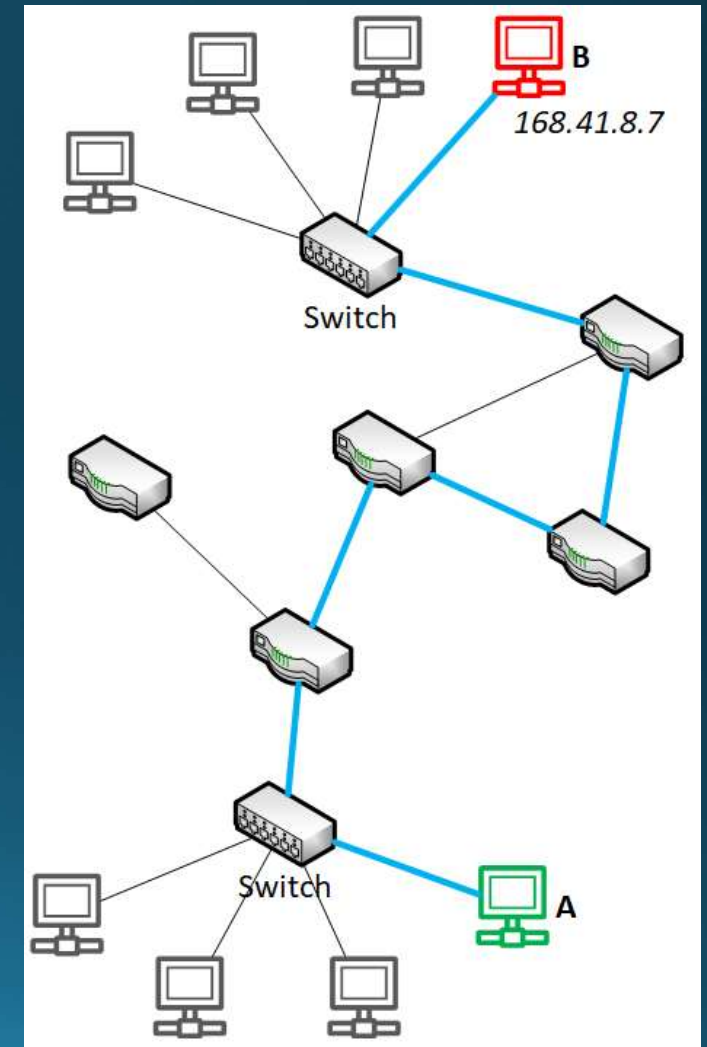
Internet Protocol

- Hvordan finder vi vej gennem netværket?
 - En "pakke" fra A til B ?
- Bruger **B's** IP-adresse
 - Dens "universelle" adresse
- Routers sørger for at viderseende
 - Hver router har en "forwarding table"
 - Matcher på IP *prefix* og vælger "næste" router



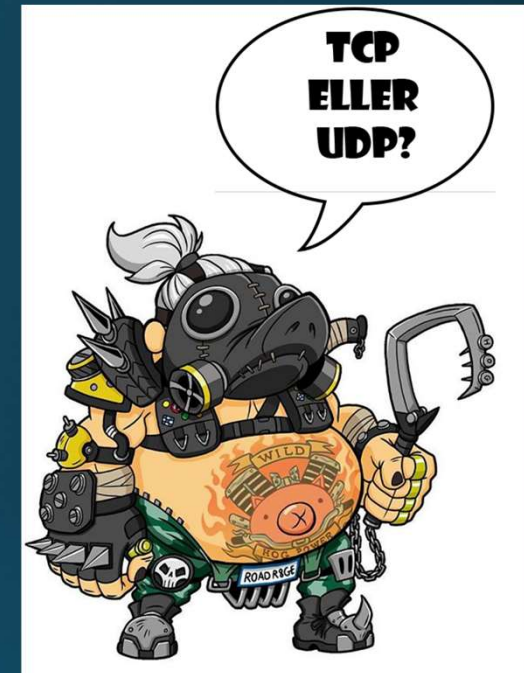
Internet Protocol

- Finder forhåbentligt frem
 - Men ikke nødvendigvis korteste rute
 - Kan dog være længere *og* hurtigere
- Alternativt går den tabt
 - TTL rammer nul
 - Router har for travlt
 - Fejl i data under transport
 - F.eks. dårligt WIFI



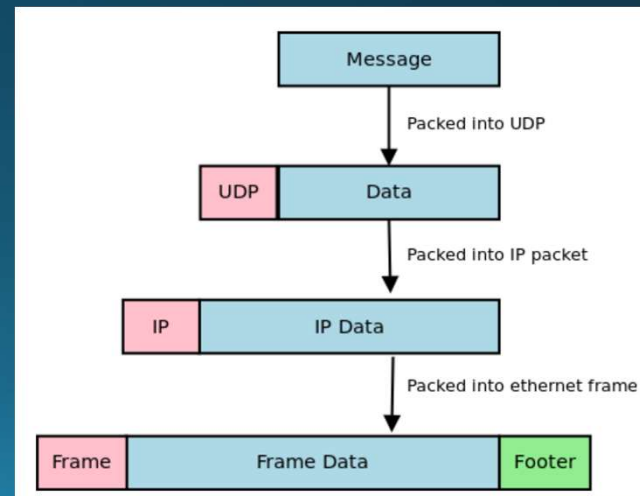
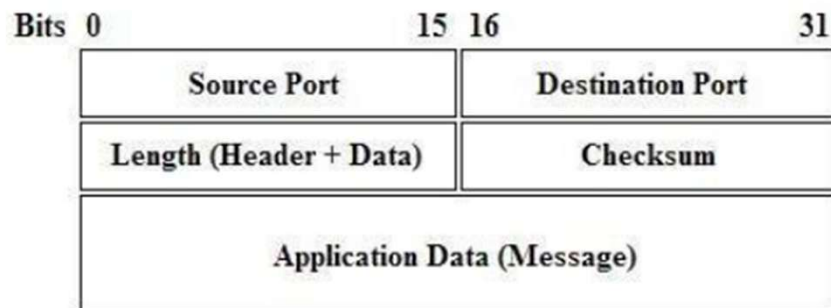
Valg af protokol

- Afhænger af hvad vi skal lave.
- Er alt data vigtigt og er rækkefølgen vigtig? – TCP
 - Overførsel af filer
 - Sende beskeder / emails
 - Lade hjemmesider
 - Non-realtime spil
 - Downloads
- Er bare det nyeste data vigtigt, og ældre data unødvendigt? – UDP
 - Streaming af video og lyd
 - Hurtige spil så som FPS



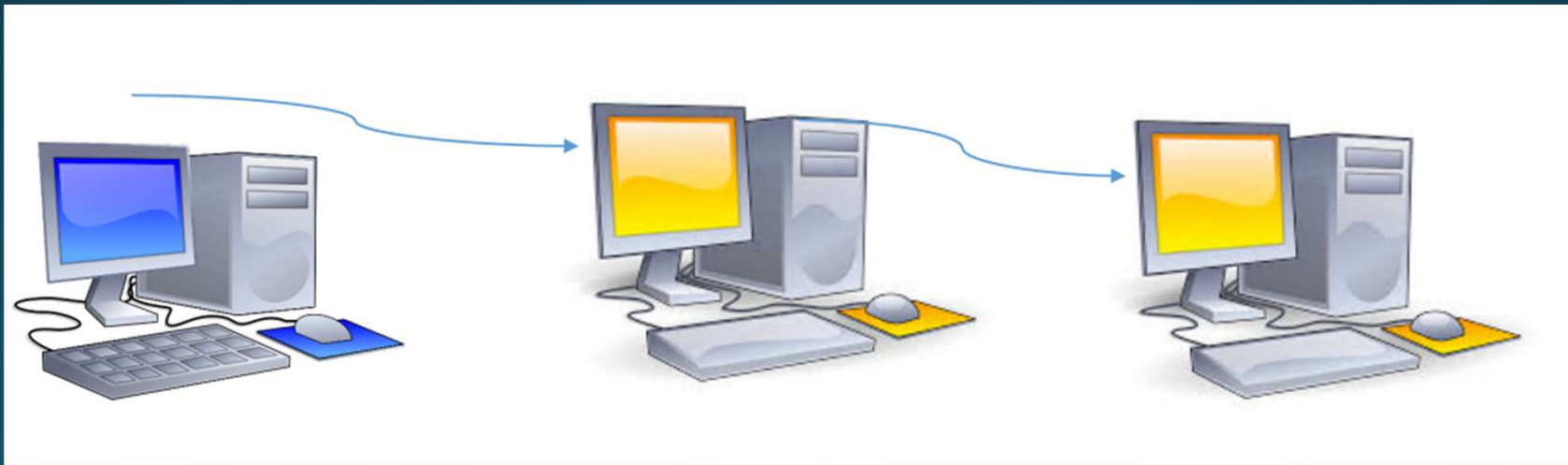
UDP

- UDP er ikke som at skrive til en fil, men som at sende en besked
 - UDP står for user datagram protocol
 - Er en anden protokol, som er bygget oven på IP, ligesom TCP
 - Har dog ikke lige så mange features som TCP
 - UDP er et meget tyndt lag ovenpå IP
 - Mindre overhead i selve pakken
 - Ca 8 bytes (UDP) vs 20-60 bytes (TCP) – dette er uden payload!



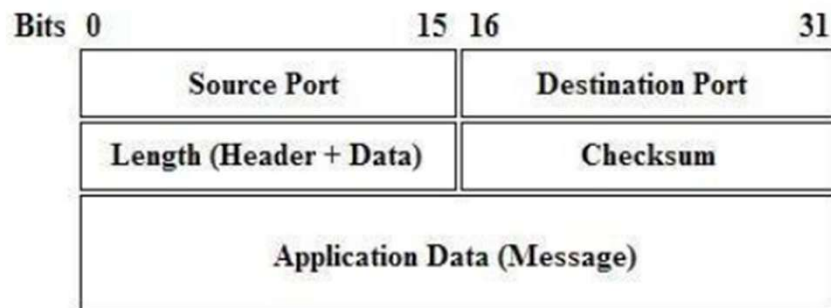
UDP

- Med UDP kan vi sende pakker til en IP adresse
 - F.eks. 112.140.20.10 Port: 52423
 - Pakken bliver sendt fra computer til computer indtil den ankommer eller går tabt



UDP

- På modtager siden skal vi bare lytte til en bestemt port, f.eks. 52423.
 - Når pakken ankommer:
 - Får vi adresse og port på afsenderen
 - Info om pakke størrelsen
 - Kan læse pakken



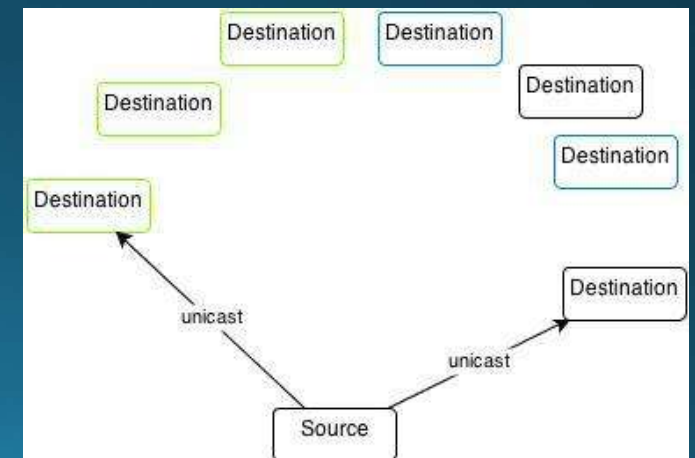
UDP

- UDP er upålidelig
 - De fleste pakker som sendes kommer igennem.
 - Men man skal forvente 1-5% pakketab
 - Afhænger af forbindelsen!
 - Der er mange computere mellem dig og destinationen, meget kan gå galt.
- Du kan heller ikke garantere rækkefølgen af pakkerne
 - 1,2,3,4,5 kan blive modtaget som 3,1,2,5,4
 - De ankommer ofte i rækkefølge, men der er ingen garanti
- Du har dog én garanti med UDP
 - Hvis du sender en pakke kan du være sikre på at:
 - At 100% af pakken ankommer eller 100% går tabt.
 - Dvs. hvis du sender 256 kb, modtager man 256 kb eller intet.
 - Det er ikke muligt at miste 3 kb, så man kun modtager 253 kb.

UDP - UNICAST

Der er kun en modtager

- UDP kan sende data på flere måde
 - UNICAST
 - Sender en besked til en specifik destination
 - Destination her betyder (IP, port)
 - Der skal laves flere unicasts hvis flere skal modtage beskeden

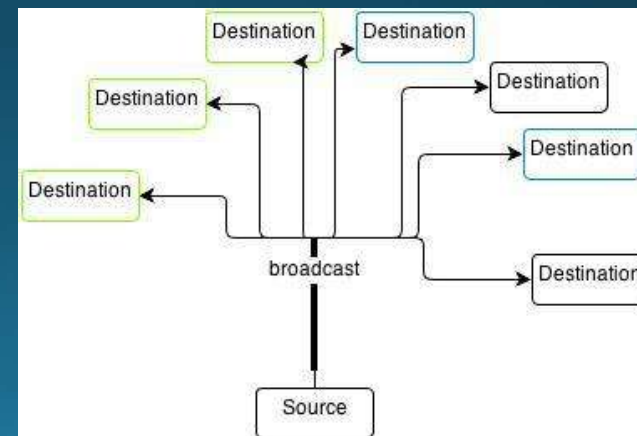


UDP - Broadcast

- UDP kan sende data på flere måde
 - Broadcast
 - Sender til alle devices på netværket
 - Bliver ikke forwarded af routers
 - For at lave et broadcast kan man sende data til en bestemt local ip-adresse
 - Så virker altså ikke over nettet...

Kan sende ud til mange modtagere

Fungerer kun på det lokale netværk forbundet af routeren

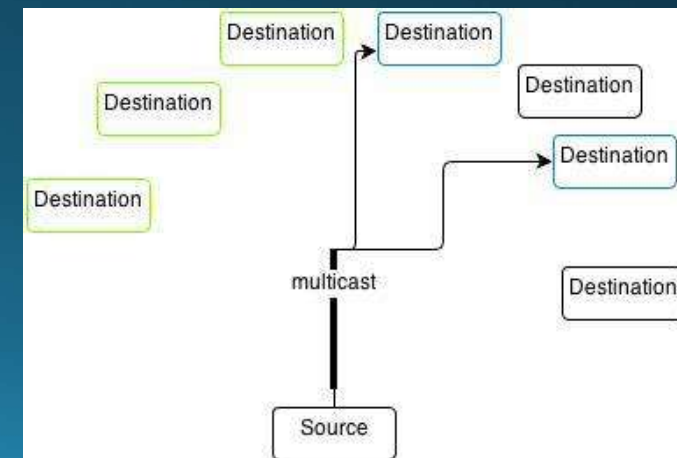


<https://www.pcmag.com/encyclopedia/term/broadcast-address>

UDP - Multicast

- UDP kan sende data på flere måde
 - Multicast
 - Sender en besked til en multicast group
 - Multicast group er ip-adresser fra 224.0.0.0 til 239.255.255.255
 - Man kan manuelt vælge at tilmelde sig en specifik multicast gruppe
 - Håndteres af routeren

```
UdpClient udpClient = new UdpClient(12500);  
udpClient.JoinMulticastGroup(IPAddress.Parse("224.1.1.1"));
```

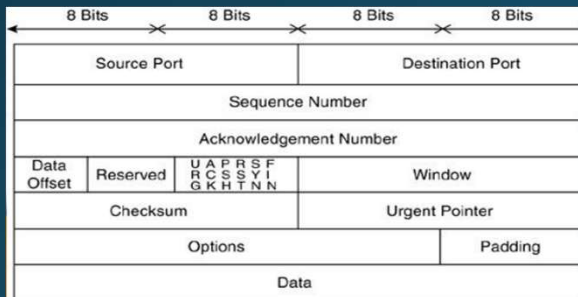


TCP vs UDP

- Skal vi så bruge TCP eller UDP?

TCP

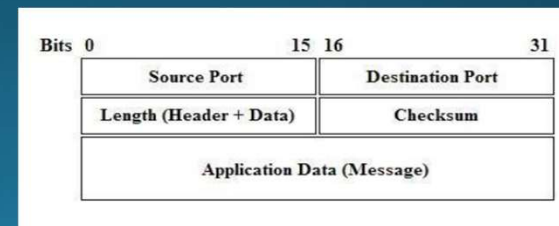
- Connection based
- Pålidelig forbindelse
- Data modtages i den korrekte rækkefølge
- Deler data op i pakker
- Sikre at data ikke sendes for hurtigt (flow control)
- Let at bruge, det er som at skrive til en fil
 - En stream af data



TCP pakke

UDP

- Den er connectionless (ingen forbindelse)
- Man skal selv kode forbindelsen hvis man har brug for den.
- Ingen garanti for at pakker leveres til modtageren.
- Pakker kan ankomme i forkert rækkefølge, være duplikeret, eller gå tabt.
- Man skal selv dele sin data op i pakker og sende dem.
- Man skal selv sikre at data ikke sendes for hurtigt.
- Man skal selv håndtere pakketab, og gendsende data hvis nødvendigt.



UDP pakke

TCP vs UDP

- Så hvad skal man vælge, og hvorfor?
 - Afhængig af use case
- TCP gør alt hvad vi har brug for
 - Den er let at bruge
- UDP
 - Mere besværlig
 - Skal selv kode ekstra funktionalitet
 - Men giver så en skræddersyet løsning
 - Optimeret ift. TCP



Kirk Bater
@KirkBater

Follow

This image is a TCP/IP Joke. This tweet is a UDP joke. I don't care if you get it.

Thread

iamkirkbater and jkjustjoshing



iamkirkbater 📷 Aug 23rd, 2017 at 9:37 AM
in #www

Do you want to hear a joke about TCP/IP?



7

7 replies



jkjustjoshing 5 months ago

Yes, I'd like to hear a joke about TCP/IP



iamkirkbater 📷 5 months ago

Are you ready to hear the joke about TCP/IP?



jkjustjoshing 5 months ago

I am ready to hear the joke about TCP/IP



iamkirkbater 📷 5 months ago

Here is a joke about TCP/IP.



iamkirkbater 📷 5 months ago

Did you receive the joke about TCP/IP?



jkjustjoshing 5 months ago

I have received the joke about TCP/IP.



iamkirkbater 📷 5 months ago

Excellent. You have received the joke about TCP/IP. Goodbye.

TCP vs UDP

FORKERT SLIDE, SKAL SLETTES

- I bør derfor bruge UDP til spil der kræver real time opdateringer.
- I bør ikke blande TCP og UDP til samme server!
 - Dvs en forbindelse til en server med udp og en anden forbindelse til en anden server via TCP er ok.
- I stedet bør i modificere UDP, så den får den funktionalitet fra TCP, som I har brug for.
- Altså lave en udvidet UDP, hvis i har brug for at vide om en klient er connectet, skal der sendes pakker frem og tilbage hele tiden.

TCP vs UDP

- Why not both?
 - Det er også en mulighed
- Formålet skal være opdelt, f.eks.:
 - TCP til, Login, Chat, Billeder og game assets
 - UDP til selve spil-opdateringer
- Kan passende håndteres af forskellige servere
 - Har I evt. prøvet at disconnecte fra et spil, men stadig kunne chatte?

UDP Send data (client)

UDP har ikke server/client forholdet ligesom TCP
Men man kan forestille sig/programmere med det som 2 forskellige pc kommunikere med hinanden

- Opretter en UDP-client til at sende og modtage UDP-pakker.
- Opretter en IP Address-instans, der repræsenterer IP-adressen for serveren, du vil kommunikere med. I dette tilfælde er det "127.0.0.1", hvilket er loopback-adressen til localhost.
- Angiver portnummeret for serveren, du vil kommunikere med. Dvs den port serveren lytter på.
- Opretter en IPEndPoint-instans, der kombinerer serverens IP-adresse og portnummer. Dette er destinationspunktet for de UDP-pakker, der skal sendes til serveren.

```
using System.Net.Sockets;
using System.Net;
using System.Text;

UdpClient udpClient = new UdpClient();
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
int serverPort = 1234;
IPEndPoint ServerEndPoint = new IPEndPoint(serverIP, serverPort);

while (true)
{
    Console.WriteLine("Skriv en besked til serveren:");
    string message = Console.ReadLine();
    byte[] sendData = Encoding.ASCII.GetBytes(message);
    udpClient.Send(sendData, ServerEndPoint);

    //Answer from server!
    byte[] receivedData = udpClient.Receive(ref ServerEndPoint);
    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Svar fra serveren: {receivedMessage} på adresse: {ServerEndPoint}");
}
```

UDP Send data (client)

- Starter en uendelig løkke for at holde klienten kørende og muliggøre interaktion med brugeren.
- Viser en prompt til brugeren for at indtaste en besked, der skal sendes til serveren.
- Læser brugerens indtastede besked fra konsollen.
 - Obs: blokende
- Omdanner den indtastede besked til et byte-array, da UDP bruger byte-data til at sende pakker.
- Sender UDP-pakken til serveren ved at kalde Send-metoden på udpClient og angive dataene og destinationspunktet.
 - Vi skal altså angive destination ved hver .Send()
 - Modsat TCP som var forbundet til en bestemt server

```
using System.Net.Sockets;
using System.Net;
using System.Text;

UdpClient udpClient = new UdpClient();
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
int serverPort = 1234;
IPEndPoint ServerEndPoint = new IPEndPoint(serverIP, serverPort);

while (true)
{
    Console.WriteLine("Skriv en besked til serveren:");
    string message = Console.ReadLine();
    byte[] sendData = Encoding.ASCII.GetBytes(message);
    udpClient.Send(sendData, ServerEndPoint);

    //Answer from server!
    byte[] receivedData = udpClient.Receive(ref ServerEndPoint);
    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Svar fra serveren: {receivedMessage} på adresse: {ServerEndPoint}");
}
```

UDP Send data (client)

- Modtager UDP-pakken fra serveren ved at kalde Receive-metoden på udpClient.
- For at modtage, skal vi angive hvem vi vil modtage fra – I dette tilfælde fra "serveren"
- OBS:
 - Vi har ikke selv åbnet en port ved UdpClient()
 - Vi får derfor tildelt en tilfældig port
 - Server kan læse "afsender" på pakke og svare til dén socket

Blokere indtil der kommer en besked

```
using System.Net.Sockets;
using System.Net;
using System.Text;

UdpClient udpClient = new UdpClient();
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
int serverPort = 1234;
IPEndPoint ServerEndPoint = new IPEndPoint(serverIP, serverPort);

while (true)
{
    Console.WriteLine("Skriv en besked til serveren:");
    string message = Console.ReadLine();
    byte[] sendData = Encoding.ASCII.GetBytes(message);
    udpClient.Send(sendData, ServerEndPoint);

    //Answer from server!
    byte[] receivedData = udpClient.Receive(ref ServerEndPoint);
    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Svar fra serveren: {receivedMessage} på adresse: {ServerEndPoint}");
}
```

UDP Send data (client)

- Konverterer de modtagne byte-data til en streng, så det kan læses og vises.
- Viser svaret fra serveren sammen med kildepunktets IP-adresse og portnummer.
- Mere info fra msdn:
 - <https://learn.microsoft.com/en-us/dotnet/framework/network-programming/using-udp-services>

```
using System.Net.Sockets;
using System.Net;
using System.Text;

UdpClient udpClient = new UdpClient();
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
int serverPort = 1234;
IPEndPoint ServerEndPoint = new IPEndPoint(serverIP, serverPort);

while (true)
{
    Console.WriteLine("Skriv en besked til serveren:");
    string message = Console.ReadLine();
    byte[] sendData = Encoding.ASCII.GetBytes(message);
    udpClient.Send(sendData, ServerEndPoint);

    //Answer from server!
    byte[] receivedData = udpClient.Receive(ref ServerEndPoint);
    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Svar fra serveren: {receivedMessage} på adresse: {ServerEndPoint}");
}
```

UDP Send data (client)

- Kan kalde “connect”
 - Så slipper vi for at angive ved hver “send”
 - Er dog nu bundet til den remote host
 - Skal stadig angive host(s) ved receive

```
using System.Net.Sockets;
using System.Net;
using System.Text;

UdpClient udpClient = new UdpClient();
IPAddress serverIP = IPAddress.Parse("127.0.0.1");
int serverPort = 1234;
IPEndPoint endPoint = new IPEndPoint(serverIP, serverPort);
udpClient.Connect(endPoint);
while (true)
{
    Console.WriteLine("Skriv en besked til serveren:");
    string message = Console.ReadLine();
    byte[] sendData = Encoding.ASCII.GetBytes(message);
    udpClient.Send(sendData);

    //Answer from server!
    byte[] receivedData = udpClient.Receive(ref endPoint);
    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Svar fra serveren: {receivedMessage} på adresse: {endPoint}");
}
```

UDP modtag data (server)

- Opretter en UDPClient-objekt og binder det til port 1234.
 - Dette er den port, som serveren vil lytte på for indgående forbindelser.
- Udskriver en besked for at angive, at serveren er klar til at modtage data.

```
using System.Net;
using System.Net.Sockets;
using System.Text;

UdpClient udpServer = new UdpClient(1234);
Console.WriteLine("Serveren er klar til at modtage data...");
while (true)
{
    IPEndPoint clientEndPoint = new IPEndPoint(IPAddress.Any, 0);
    byte[] receivedData = udpServer.Receive(ref clientEndPoint);

    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Modtaget: {receivedMessage} fra {clientEndPoint}");

    string responseMessage = "Tak for beskeden!";
    byte[] responseData = Encoding.ASCII.GetBytes(responseMessage);
    udpServer.Send(responseData, clientEndPoint);
}
```

UDP modtag data (server)

- Starter en evig løkke, der vil køre kontinuerligt.
- Opretter en IPEndPoint til at gemme klientens adresse og port.
 - IPAddress.Any angiver, at serveren vil lytte på alle tilgængelige netværksgrænseflader
 - 0 angiver at klientens port er ukendt pt, og alle er porte er gode.
- udpServer.Receive
 - Modtager data fra klienten og gemmer det i "receivedData".
 - OBS: blokende.
- "ref clientEndPoint" angiver, at klientens adresse og port også skal gemmes i clientEndPoint-variablen.
 - Dvs nu er IP og port kendt fra klienten og gemt i clientEndPoint

```
using System.Net;
using System.Net.Sockets;
using System.Text;

UdpClient udpServer = new UdpClient(1234);
Console.WriteLine("Serveren er klar til at modtage data");
while (true)
{
    IPEndPoint clientEndPoint = new IPEndPoint(IPAddress.Any, 0);
    byte[] receivedData = udpServer.Receive(ref clientEndPoint);

    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Modtaget: {receivedMessage} fra {clientEndPoint}");

    string responseMessage = "Tak for beskeden!";
    byte[] responseData = Encoding.ASCII.GetBytes(responseMessage);
    udpServer.Send(responseData, clientEndPoint);
}
```

UDP modtag data (server)

- Konverterer de modtagne data fra bytes til en streng ved hjælp af ASCII-kodning.
- Udskriver den modtagne besked sammen med klientens adresse og port.
- Opretter en svarbesked, der skal sendes til klienten.
- Sender svarbeskeden til klienten ved hjælp af klientens adresse og port, der er gemt

```
using System.Net;
using System.Net.Sockets;
using System.Text;

UdpClient udpServer = new UdpClient(1234);
Console.WriteLine("Serveren er klar til at modtage data...");
while (true)
{
    IPEndPoint clientEndPoint = new IPEndPoint(IPAddress.Any, 0);
    byte[] receivedData = udpServer.Receive(ref clientEndPoint);

    string receivedMessage = Encoding.ASCII.GetString(receivedData);

    Console.WriteLine($"Modtaget: {receivedMessage} fra {clientEndPoint}");

    string responseMessage = "Tak for beskeden!";
    byte[] responseData = Encoding.ASCII.GetBytes(responseMessage);
    udpServer.Send(responseData, clientEndPoint);
}
```

UDP send/modtag data i aktion

Tildelt port: 54277

```
Skriv en besked til serveren:  
hej server fra guy 1  
Svar fra serveren: Tak for beskeden! på adresse: 127.0.0.1:1234  
Skriv en besked til serveren:
```

Lytter på 1234

```
Serveren er klar til at modtage data...  
Modtaget: hej server fra guy 1 fra 127.0.0.1:54277  
Modtaget: hej server fra guy 2 fra 127.0.0.1:54278  
|
```

Tildelt port: 54278

```
Skriv en besked til serveren:  
hej server fra guy 2  
Svar fra serveren: Tak for beskeden! på adresse: 127.0.0.1:1234  
Skriv en besked til serveren:
```

Øvelse UDP – Heartbeat

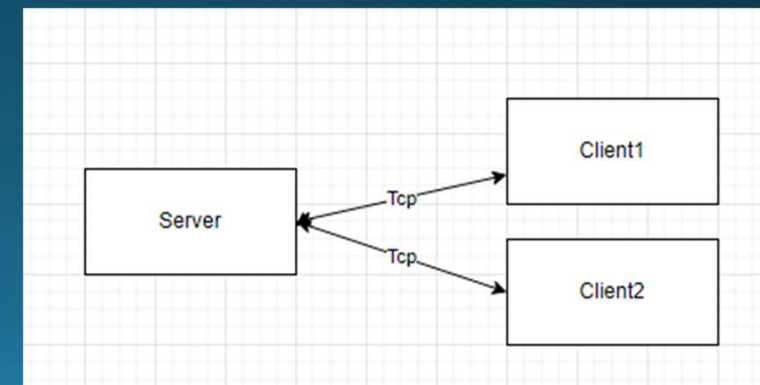
- Der er et UDP projekt på moodle
 - Client kan sende besked til server og modtage et svar
- Serveren skal “holde øje” med om en client stadig er forbundet
 - Selvom vi som brugere ikke sender beskeder
 - Hvis klienten lukkes, skal serveren skrive “client disconnected” til konsollen
- Men UDP er “connectionless”, så hvordan?
 - Vi skal have en “heartbeat funktion” **skal sende en besked som viser, at den er fortsat forbundet**
 - Send besked hver n sekunder og sig om du stadig er der
 - F.eks.: While løkke med sleep eller System.Timers kan kalde funktion periodisk
- Serveren skal altså holde høje med denne for at etablere et disconnect
 - Hvor længe skal der gå?
 - Hvad hvis en besked pludselig dukker op igen?
 - Sendes der begge veje?

Det hele starter med snyd

- Når vi skal lave et multiplayer spil er vi nødt til at starte med at kigge på det værste tilfælde.
- Laver man et single player spil, betyder snyd ikke meget
 - Det er spilleren selv det går udover.
- Laver man multiplayer, og en spiller snyder, går det ud over alle spillere i sessionen.
- Flere ting kan gøres for at undgå snyd i multiplayer spil, men den overordnede regel der gælder, både for hurtige såsom langsomme er:
 - DONT TRUST THE PLAYER(client).
 - De snyder!

Topologier – Client/server

- Hver client opretholder én forbindelse til serveren.
- Clienter har ingen direkte forbindelse, udover gennem serveren
- Serveren holder styr på spillet regler, spillere visualisere blot dette.
- Hvis en client agerer som server er dette stadig client/server, da den også kan spille selv.

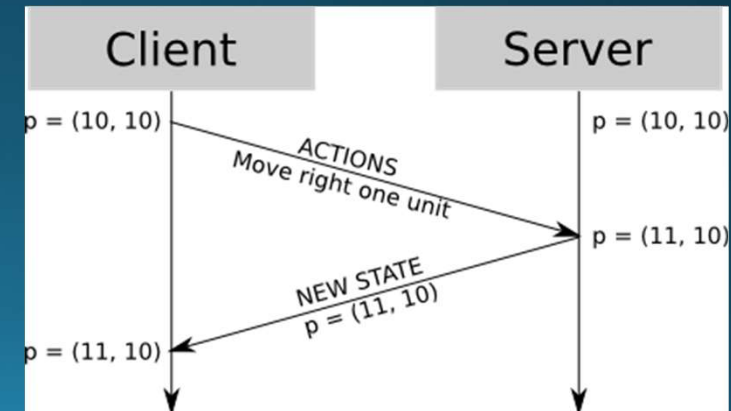


Authoritativ server og dumme klienter

- Brugen af en autoritativ server og dumme klienter er en almindelig tilgang i multiplayer-spil.
- Klienter sender input til serveren, som behandler og opdaterer spilstilstanden.
- En autoritativ server er en central server, der har fuld kontrol over spilstilstanden.
- Serveren er den eneste autoritet og træffer beslutninger om alt, der sker i spillets verden.
- En vigtig fordel ved en autoritativ server er beskyttelse mod snyd.
- Klienten kan ikke manipulere værdier som stats eller position, da serveren har det endelige ord.

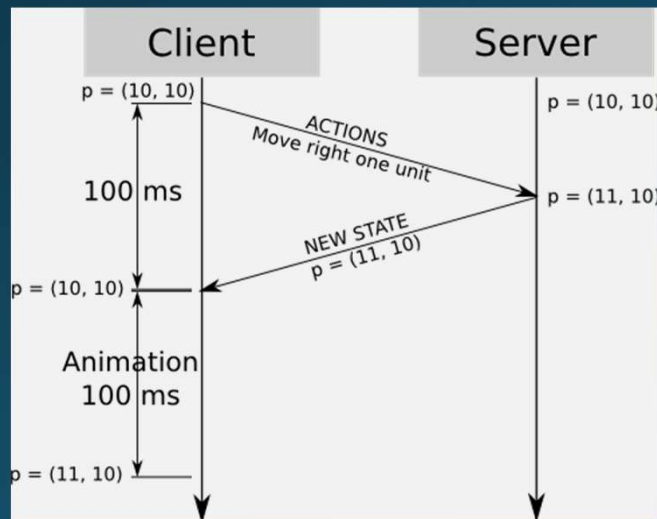
Authorativ server og dumme klienter

- Lader vi spilleren selv holde styr på sin position, kan vi nemt bare sige jeg er på (20,10)
 - Også selvom denne position er urealistisk for spil stadiet
 - Dette går i mod ideen om en authorativ server, og bør derfor undgås.
- Sender vi i stedet blot forespørgslen, "move right" er det nu op til serveren at opdatere denne position og sende tilbage til spilleren.
- Dette hjælper selvfølgelig på spillet stadiet, men der er stadig problemer...
 - Det går laaaangsomt.
 - Spillere hader latency.

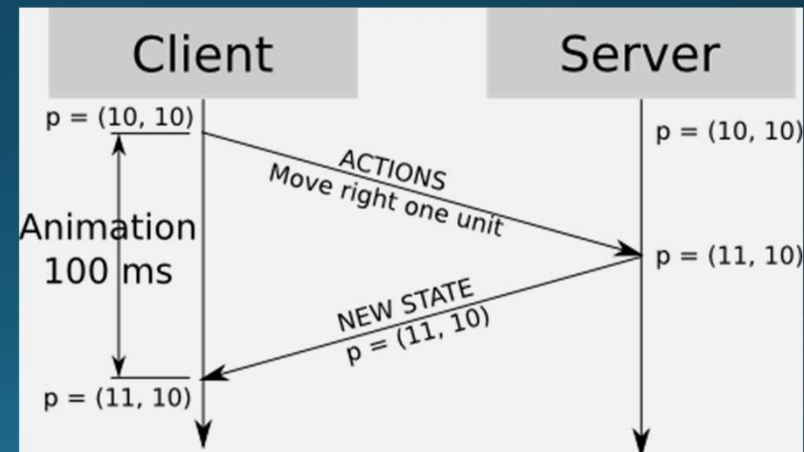


Clientside Prediction

- Konceptet er simpelt, både klient og server udfører opgaven, og hvis der er fejl rettes der ind efter hvad Serveren siger.
- Ved antagelse af 100ms lag og en animation på 100ms kan vi få følgende:



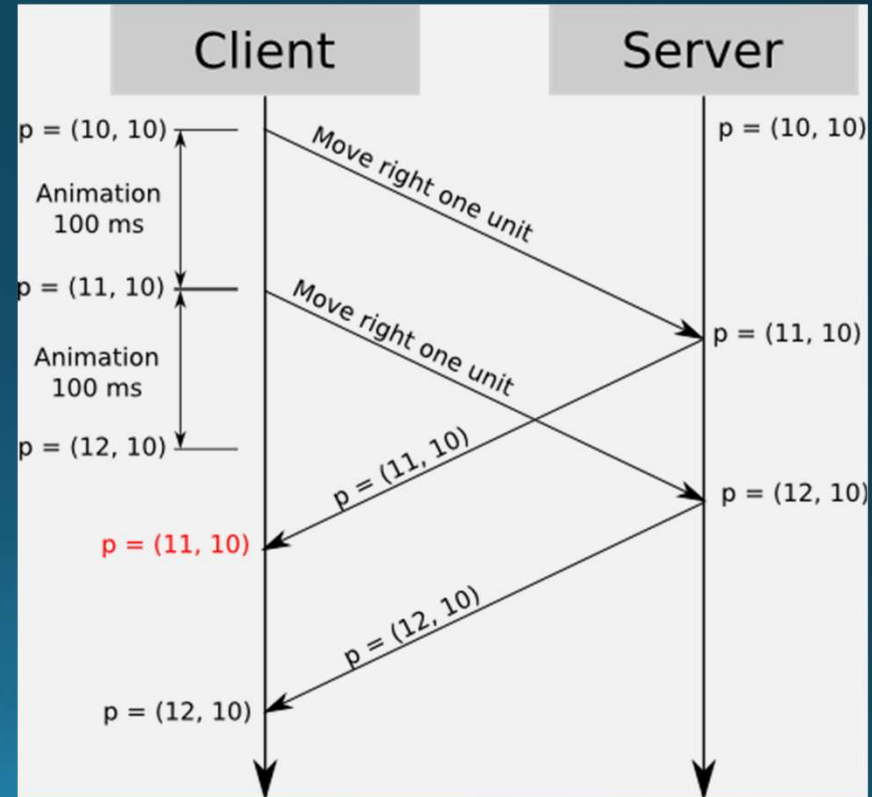
Uden clientside prediction



Med clientside prediction

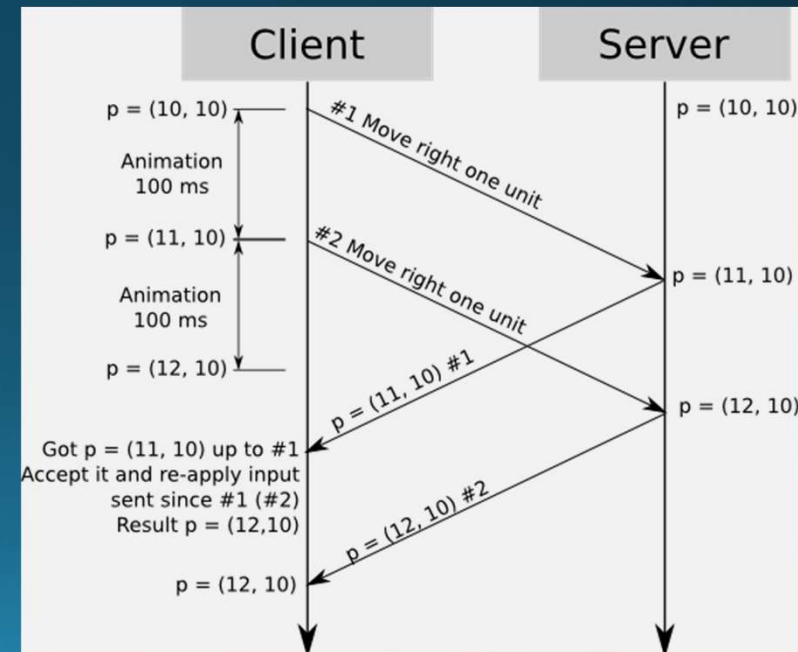
Synkroniserings problem

- Tidligere eksempel virkede kun fordi tallene gik op. Lad os prøve uden de gør:
- Vi har nu lag på 250ms (125ms hver vej)
 - Stadig 100ms animation
- Vi løber nu ind i et problem!
 - Ved $t = 250$ har client pos på (12,10)
 - Modtager (11,10) som korrekt pos fra server
 - Hopper derfor tilbage
 - Ved $t = 350$ modtages (12,10)
 - Hopper derfor frem igen
- Det vil hverken føles godt eller ser godt ud.



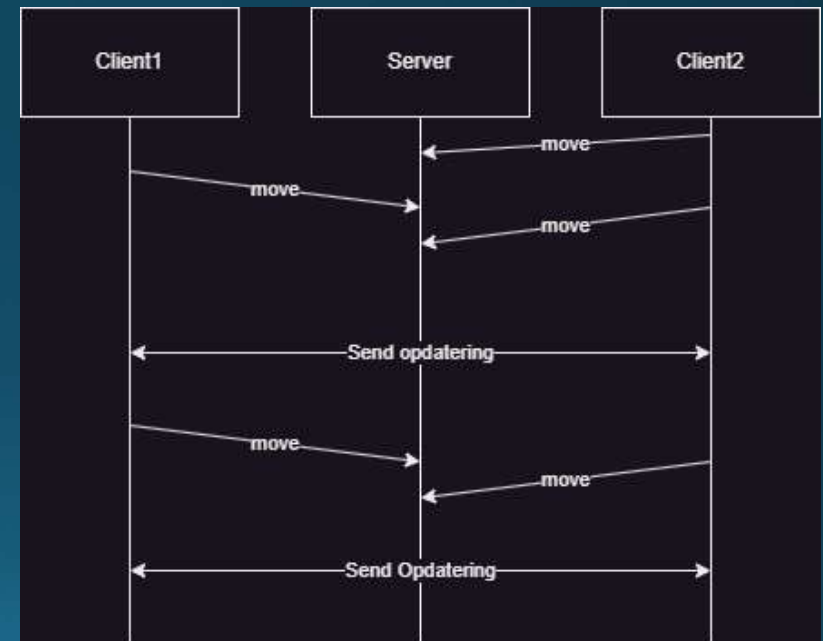
Server reconciliation

- Løsningen til dette problem er at huske på at spilleren ser spillet i Nutid, hvorimod serverens opdateringer er i datid, grundet lag!
- Der er heldigvis en simpel løsning til dette ved at indeksere input der bliver sendt til serveren
 - Ved $t=250$, siger server fra #1 at vi skal være på (11,10)
 - Som klient ved vi at 1# Er blevet godkendt, men 2# er sendt, men ikke godkendt
 - Vi sætter derfor vores position til resultat sidst godkendte server pakke(1#) + alle endnu ikke processerede #2, vores pos ender derfor på (12,10)
 - Ved $t=350$ modtages #2 svar, som stadig er korrekt og er nu tilbage ved start. (ingen # mangler svar)



Server Time Step

- I vores tidligere eksempel tog serveren blot input fra klienten og sendte dem ud så hurtigt som de kom ind.
- Dette skalere ikke så godt med flere spillere, hvis hele verden bliver opdateret hver gang et input bliver modtaget.
- I stedet kan man vælge at lade verden opdatere med et fast interval f.eks 10/s eller 60/s (60 hz)
- Dvs. serveren modtager opdateringer fra klienter når de laver dem, men queue'er dem op indtil næste timestep og sender dernæst ud til alle klienter.



Snapshotting

- Snapshotting er en metode til at opretholde konsistens mellem replikerede systemer ved at tage periodiske "snapshots" af systemets(serveren) tilstand og overføre dem til replikaer(clienter).
- Med snapshotting modtages der aktioner fra klienter som bliver brugt til at opdatere systemets tilstand.
- Klienter sender altså inputs og modtager og reagerer på opdateringer
- Systemets tilstand bliver derefter sendt ud til alle klienter ved et fast interval.
 - F.eks bruger team fortress 2 en rate(hz) på 66
 - Mens left 4 dead 2 bruger 30hz

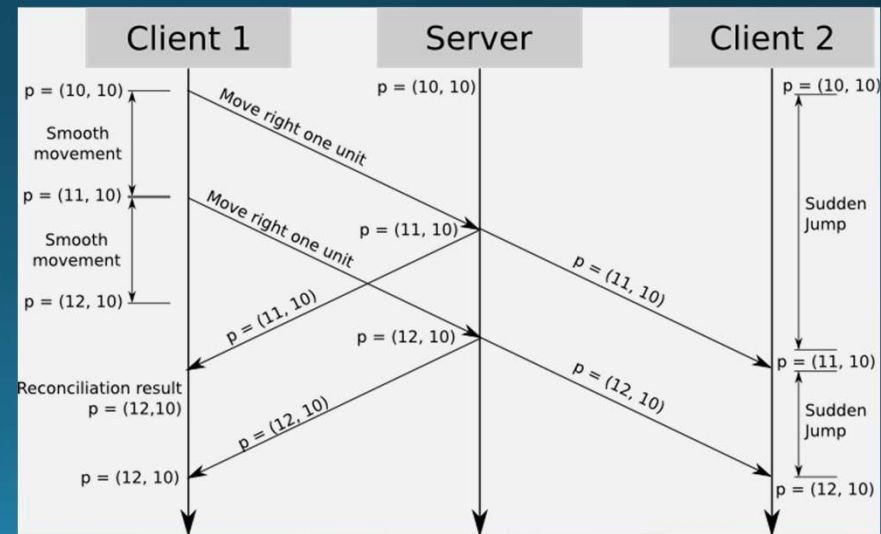
Snapshotting og udp

- Vi ved at UDP er en connectionless protocol.
 - Pakker kan komme i vilkårlig rækkefølge
 - Pakker kan gå tabt
 - Pakker kan være dublikerede
- Dette spiller stærkt sammen med snapshotting.
 - Giver vi vores snapshots et sekvens nummer, kan vi:
 - Smide en gammel pakke væk blot ved at sammenligne sekvens nummer
 - Smide en duplikeret pakke væk
 - Pakketab betydere mindre, når vi sender ofte
- Hvis der virkelig er behov for det kan vi implementere dele fra TCP til vores UDP løsning.
 - Se mere her: https://gafferongames.com/post/reliable_ordered_messages/

Langsomme opdateringer

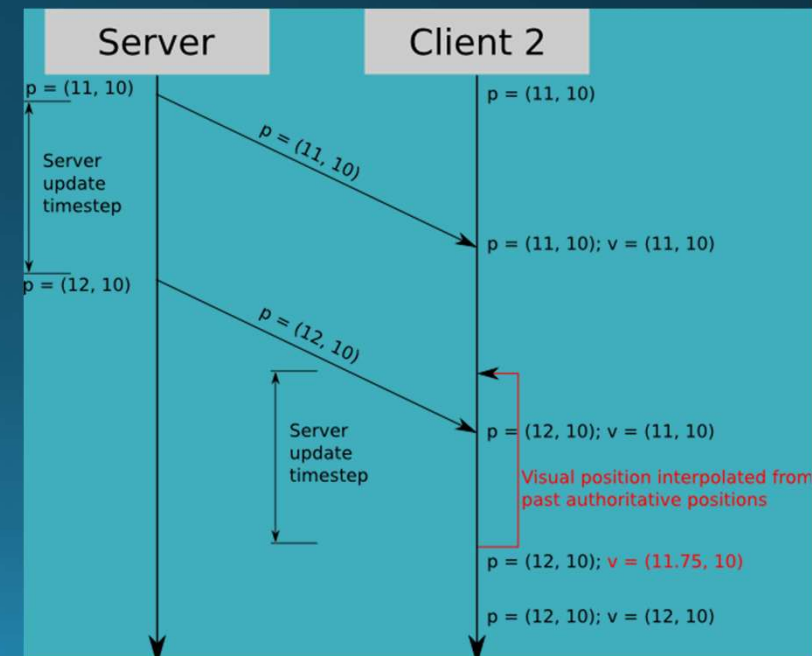
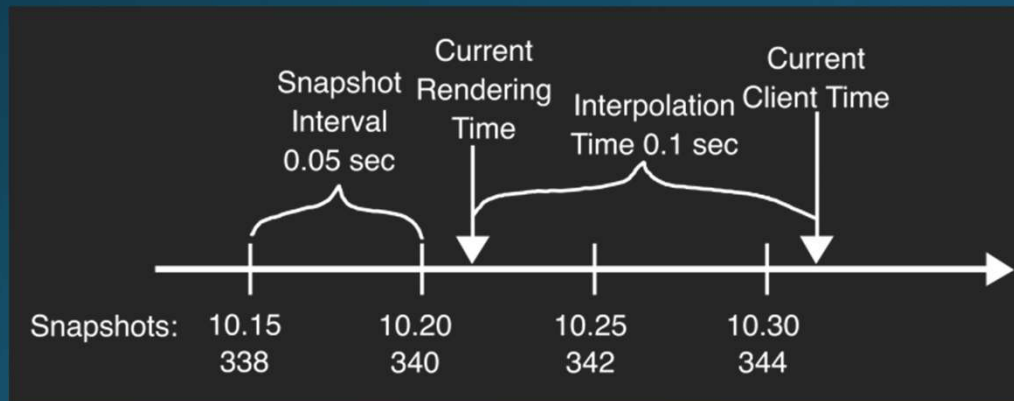
- Clientside prediction gør at spillet virker fint for spilleren der sender opdatringer på serveren
- Det samme kan ikke siges for andre connectede klienter
- Det bliver meget choppy, hvis der blot opdateres ved nye server shapshots.
- Essentielt teleporter spillere rundt!
 - Dog med et fast interval = server opdaterings hz

https://gafferongames.com/post/snapshot_interpolation/



Entity Interpolation

- Løsningen er at lade andre spillere (eller entiteter) være i fortiden.
- Her gemmes en håndfuld tidligere modtaget position serveren
- Og i stedet for blot at sætte en spiller til en position udføres lineær interpolation mellem tidligere godkendte positioner.
- Vi udregner altså positioner mellem serverpakker
 - Disse udregnes baseret på tidligere godkendte positions.



Entity Interpolation

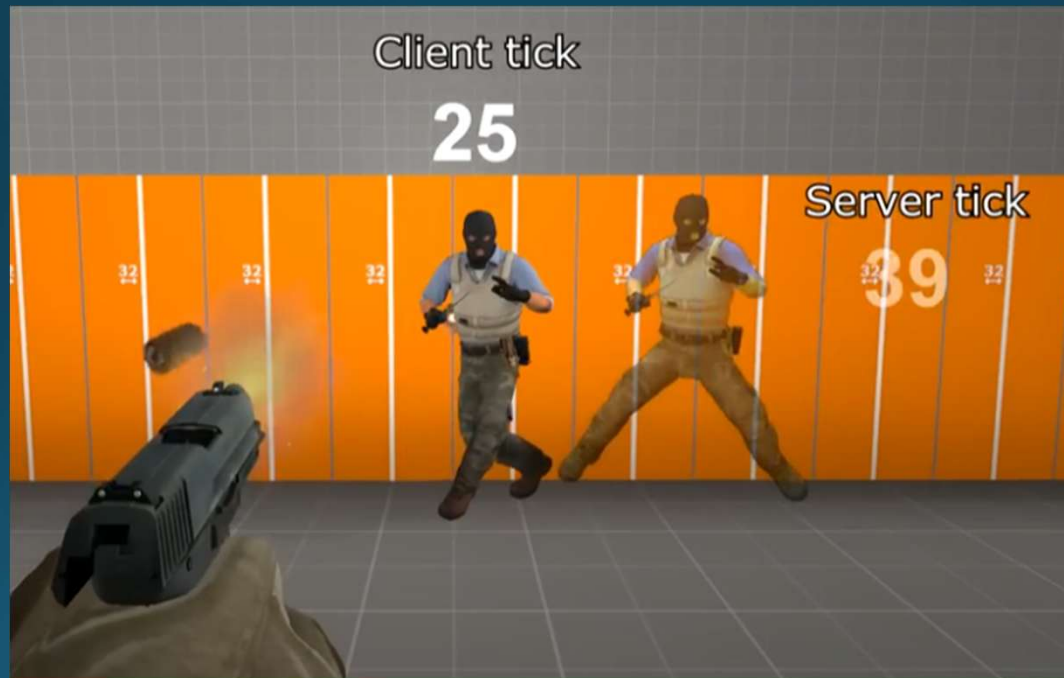


Recap af struktur

- Serveren modtager input fra alle klienter med timestamps.
- Serveren behandler input og opdaterer spillets state.
- Serveren sender regelmæssige Snapshots af spillets state til alle klienter.
- Klienter sender input og simulerer deres verdener lokalt.
- Klienter modtager opdateringer om spillets state.
 - Synkroniserer forudset tilstand med autoritativ tilstand.
 - Interpolerer kendte tidligere tilstande for andre enheder.
- Fra en spillers synspunkt har dette to vigtige konsekvenser:
 - Spilleren ser sig selv i nutiden.
 - Spilleren ser andre enheder i fortiden.
- Dette fungerer fint til nogle situationer, men ikke til alle. F.eks ikke godt til tidssensitive events såsom at skyde!

Lag compensation

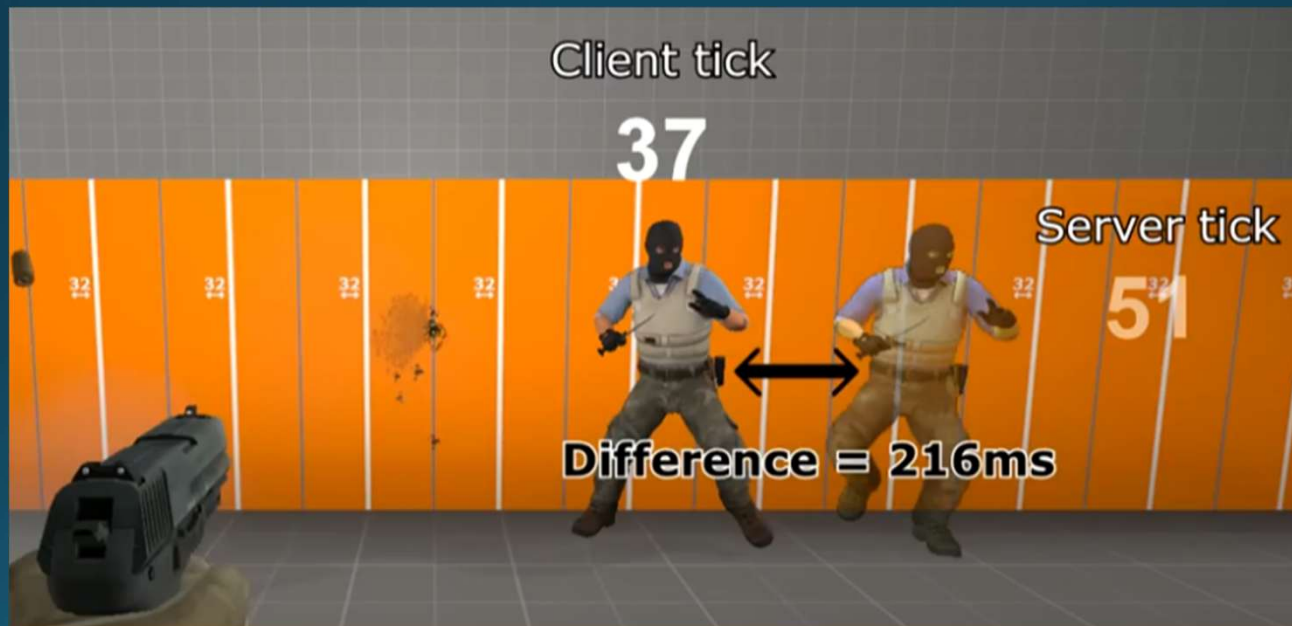
- Vi skyder, når vi ser en enemy



<https://www.youtube.com/watch?v=6EwaW2iz4iA>

Lag compensation

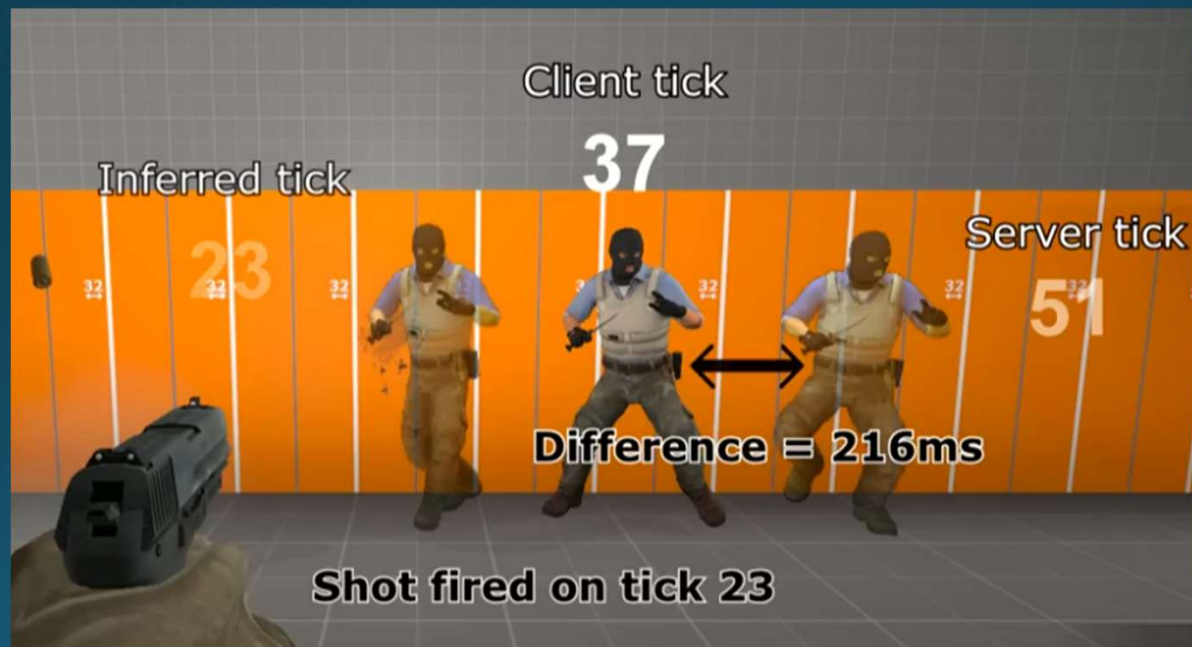
- Enemy flytter sig, inden vores besked når op til serveren



<https://www.youtube.com/watch?v=6EwaW2iz4iA>

Lag compensation

- Server ser på latency, og ruller alle player tilbage
 - Ser om skuddet ville have ramt



<https://www.youtube.com/watch?v=6EwaW2iz4iA>

Øvelse – “Forbedret” UDP!

- Server til rådighed
 - Lytter på port 21000
 - Forventer ASCII format **ASCII**
 - Send “@[name]”
 - (starter et ny “session”)
 - I modtager én besked med en (tilfældig) string
- Send så dén string tilbage
 - Én char ad gangen
 - I formatet *[index][character]*
 - F.eks.: “s8A3k9NL1PKu”
 - → (F.eks. “0s”, “18”, “2A”, ..., “11u”)
- Ikke alle pakker når frem!
 - I får konstant “status” fra server i form af bitfield (“0011001001101”)
 - Gen-send “tabte” beskeder
- Server timeout
 - Ved ingen (nye) beskeder efter et “stykke tid”
 - Start da forfra (ny “session”)

Øvelse – “Forbedret” UDP!

- Målet
 - (Gen)send indtil alle beskeder er modtaget af serveren
 - Fuldt bitfield fra serveren
 - Rækkefølge er ikke vigtig (det kan sorteres af modtageren)
 - Se hvor hurtigt i kan gøre det
 - Der bliver taget tid
 - Undgå at “spamme” beskeder
 - Gensend kun når nødvendigt
 - Send højst 4 beskeder i sekundet
- Kort demo...

Prediction + Reconciliation

UDP SpilØvelse

Øvelse Fixing af realtid – RØD

- På Moodle ligger 2 projekter (UDPGame – Monogame)
 - En consol app – serveren
 - En monogame app – clienten
 - Gider den ikke virke, så blot kopier alt over i et nyoprettet monogame projekt
- Det er baseret på gambriel gambetta's eksempel på hans site.
 - I kan derfor bruge denne til facit liste 😊
 - <https://www.gabrielgambetta.com/client-side-prediction-live-demo.html>
 - Projektet indeholder kode til at have en helt basal snapshotting gameserver. Clienten er i lockstep, hvilket vil sige at den kun bevæger sig ved svar fra serveren.
 - Dette fungerer jo ikke så godt med latency.

Øvelse Fixing af realtid – Del 1

- Udvid programmerne til at have
 - Clientside prediction (implementeres først)
 - Server reconciliation (implementeres ovenpå, efterfølgende)

Øvelse Fixing af realtid – Del 2

- Udvid programmerne til at serveren kan have flere en enkelt klient.
 - Klient 2 kan godt blot være en simpel spectator, så den kan se hvad der foregår, men ikke udføre inputs.
- Udvid dernæst klienten til at gøre brug af interpolation så spillet ser okay ud på client 2, selv hvis der er latency.
- Bonus:
 - Udvid til at have bevægelse i y-aksen også
 - Gør det muligt for begge spillere at bevæge hver deres bold
- Dagen i morgen også afsat til øvelse
 - Enten denne, eller afslutning af tidligere øvelser
 - Se også opsummerings videoer for netværks koncepterne på moodle